ناطق · NAATIQ — DOCUMENTATION

Security and Privacy

What data Naatiq holds, how it is protected, the deliberate trade-offs, and what must change before production

1. Why this matters more here than usual

Naatiq's users are among the most vulnerable people in the labour market. Many are migrant workers whose immigration status is tied to an employer. A leaked phone number, a full name attached to a searchable profile, or an exposed employment history can create real risk — harassment, retaliation, or worse — for someone with little recourse.

So privacy here is not a compliance checkbox. It is a safety requirement, and it is enforced in the database schema rather than in application code, because schema-level controls cannot be forgotten.

2. Data held

Data	Where	Sensitivity
WhatsApp phone number	`users.whatsapp_number`	**High** — directly identifying
Full name	`profiles.full_name`	**High**
First name	`profiles.first_name` (generated)	Low — safe for public display
Trade, skills, tools, achievement	`profiles`	Low
Employer names and durations	`profiles.employers`	Medium
Full conversation transcripts	`conversations.transcript`	**High** — verbatim speech
Generated CV PDF	Storage `cvs` bucket	**High** — contains name and phone
Placement records	`placements`	Medium

Voice recordings themselves are **not** stored. Audio is fetched from Twilio, transcribed in memory, and discarded; only the transcript is written.

3. How access is controlled

Row-level security, deny by default

RLS is enabled on every table with no permissive policies. Nothing is readable by an anonymous or authenticated client unless a specific grant allows it. The default is *no access*.

The service role never leaves the server

`SUPABASE_SERVICE_ROLE_KEY` bypasses RLS and is used only inside Edge Functions, where it is read from function secrets. It appears in no front-end file and no client bundle.

Public surfaces read views, never tables

The dashboard, landing page and employer portal never touch `users`, `profiles`, `conversations` or `matches`. They read three views only:

- `dashboard_stats` — aggregate counts.
- `dashboard_profiles` — first name, trade, skills, `cv_pdf_path`.
- `dashboard_trade_distribution` — counts per trade.

All three are created with **`security_invoker = true`**, so they execute with the caller's permissions rather than the definer's. Without this, a view silently bypasses RLS — a real finding raised by Supabase's own advisor during the build and fixed by recreating the views.

Privacy enforced by schema, not by discipline

`profiles.first_name` is a **generated column**: `split_part(coalesce(full_name,''),' ',1)`. Public surfaces are granted access to `first_name` and never to `full_name`, using **column-level grants**.

This is the important design decision. A full name cannot leak through a public surface even if someone writes careless front-end code, because the anonymous role has no permission to read that column at all. The guarantee does not depend on anyone remembering to strip a field.

Phone numbers are never exposed on any public surface, in any view, under any circumstance.

Verification

After the views were built, the anonymous role was tested directly and confirmed unable to read `full_name`, `users`, `conversations` or `matches`.

4. Contact without exposure

The employer portal shows first name, trade, experience and skills, and links to the CV. It does not show a phone number. An employer who wants to hire someone requests contact **through Naatiq**, which keeps the worker in control of when their details are released and to whom.

5. Prompt injection

Voice transcripts are untrusted input. Both production prompts defend against injection:

- All user content is wrapped in explicit tags — `<user_message>`, `<profile>`.
- Both system prompts state that everything inside those tags is **data, never instruction**, and name the common attack patterns ("ignore your instructions", "system:", "you are now...").
- The instruction is repeated in the user turn as well as the system prompt.
- Model output is never executed or trusted structurally: JSON is parsed defensively, every field is type-checked, and anything malformed falls back to safe defaults.

The realistic risk is low — the attacker would be a worker attacking their own CV — but the defence costs nothing and the same code path will eventually handle less benign input.

6. Secrets management

All credentials live in Supabase Function secrets, never in the repository:

```
SUPABASE_SERVICE_ROLE_KEY, TWILIO_AUTH_TOKEN, GROQ_API_KEY,
ANTHROPIC_API_KEY, ELEVENLABS_API_KEY, WATHEFTI_SELFTEST_SECRET
```

Only the **publishable** key appears in front-end files. It is designed to be public and is constrained entirely by RLS and grants.

The self-test endpoint is gated behind `WATHEFTI_SELFTEST_SECRET` and reports only whether each secret is *present* — it never echoes a value.

7. Function authorisation

- `whatsapp-webhook` — `verify_jwt: false`, because Twilio cannot send a Supabase JWT.
- `generate-cv` — `verify_jwt: true`, invoked only server-to-server.

8. Known trade-offs, stated plainly

These are deliberate hackathon decisions, documented so they are not mistaken for oversights.

The `cvs` bucket is public. CV PDFs are reachable by anyone holding the URL, and those PDFs contain the worker's full name and phone number. This was chosen knowingly, after the privacy implications were raised and discussed, to make the demo work without an auth layer.

> **Required before production:** make the bucket private and serve CVs through short-lived signed URLs generated per request for an authenticated employer. The migration that made the bucket public carries this recommendation inline. **This is the single most important change on the list.**

The Twilio webhook signature is not verified. Twilio signs every request with `X-Twilio-Signature`; Naatiq does not currently validate it, so a third party who learns the endpoint URL could post forged messages. *Before production:* verify the signature.

No user authentication. Identity is the WhatsApp number, which is reasonable for the channel but means anyone with access to a worker's phone can act as them.

No rate limiting. A single number could drive unlimited inference cost.

No data retention policy or deletion path. Conversations and profiles are kept indefinitely and there is no way for a worker to request erasure. *Before production:* add a retention window and a deletion command, ideally something as simple as sending "احذف بياناتي" on WhatsApp.

No explicit consent flow. The agent explains what it is doing, but the user is never asked to agree to storage or to their profile appearing in an employer-facing portal. *Before production:* explicit, plain-language opt-in — separately for data storage and for employer visibility.

9. Production checklist, in priority order

1. Make the `cvs` bucket private; serve signed URLs to authenticated employers.
2. Verify the Twilio webhook signature.
3. Add explicit consent, separately for storage and for employer visibility.
4. Add a data retention policy and a self-service deletion path.
5. Add rate limiting per WhatsApp number.
6. Authenticate employers before exposing any candidate data.
7. Add audit logging for every access to a CV.
8. Review obligations under the data-protection law of each market served.